

Theory of Computation

ALPHABETS :-

An alphabet is a finite, non-empty set of symbols.

We use the symbol Σ for an alphabet

e.g.

$\Sigma = \{0, 1\}$ is the binary alphabet.

STRING

A string (or sometimes word) is a finite sequence of symbols chosen from some alphabets.

e.g.:- 10010 is a string from the binary alphabet $\Sigma = \{0, 1\}$

EMPTY STRING

Empty string is the string with zero occurrences of the symbols denoted by ϵ .

LENGTH OF STRING

The no. of symbols in the string

Notation for length of string w is $|w|$

← cardinality.

$$|011| = 3$$

$$|\epsilon| = 0$$

The set of all strings over an alphabet Σ is denoted by Σ^*

$$\{0, 1\}^* = \{\epsilon, 0, 1, 01, 10, 11, 000, \dots\}$$

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \dots$$

$$\Sigma^* = \Sigma^+ \cup \{\epsilon\}$$

$$\Sigma^+ = \Sigma^* - \{\epsilon\}$$

* Zero or more occurrence

Σ finite set

Σ^*, Σ^+ — infinite

+ one or more.

e.g.:-

$$w = abqa \\ = L_1$$

$$v = baqa \\ = L_2$$

a, b, c alphabet
0, 1, 2
 $uvw \rightarrow$ strings

The concatenation of two strings w and v is the string obtained by appending the symbol of v to the right end of w

$$wv = abaqbaqa$$

Reverse of String

if $w = a_1 a_2 \dots a_n$

denoted by w^R

$$w^R = a_n \dots a_2 a_1$$

Language

Language is denoted by L

complement of Language L is

$$\bar{L} = \Sigma^* - L$$

Reverse of a language is the set of all string reversals.

$$L^R = \{w^R : w \in L\}$$

concatenation of two languages.

L_1 and L_2

$$L_1 L_2 = \{ \underbrace{xy}_{\substack{\text{word or} \\ \text{string}}} : \overset{\text{belongs}}{x} \in L_1, y \in L_2 \}$$

Star closure

$$L^* = L^0 \cup L^1 \cup L^2 \dots \quad L^0 = \{\lambda\}$$

← empty string

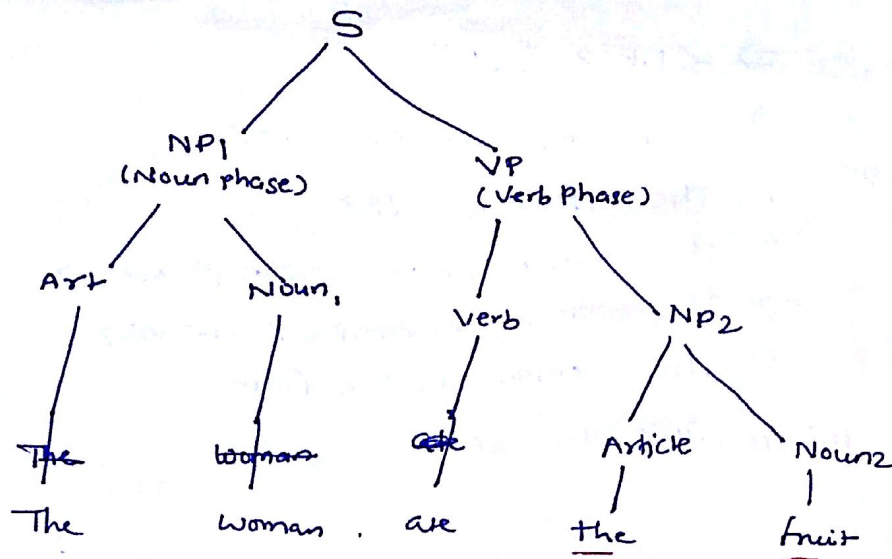
* - zero or more

Positive closure

$$L^+ = L^1 \cup L^2 \dots$$

+ - one or more

Parse Tree or Syntax tree



We do parsing to check whether the sentence is according to the rules of English grammar.

Set of rules

$\langle S \rangle \rightarrow \langle NP1 \rangle \langle VP \rangle$
 $\langle NP1 \rangle \rightarrow \langle Article \rangle \langle noun1 \rangle$
 $\langle VP \rangle \rightarrow \langle Verb \rangle \langle NP2 \rangle$
 $\langle NP2 \rangle \rightarrow \langle Article \rangle \langle Noun2 \rangle$
 $\langle Art \rangle \rightarrow The$
 $\langle Noun1 \rangle \rightarrow woman$
 $\langle verb \rangle \rightarrow ate$
 $\langle noun2 \rangle \rightarrow fruit$

Production
rules

Or

Rules

$N \rightarrow$ set of non-terminals
 $T \rightarrow$ set of terminals
 $S \rightarrow$ start symbol
 $V \rightarrow$ Total Alphabet

S is special Non-Terminal

$$V = N \cup T$$

Non-Terminal nodes are those nodes from which you can derive nodes.
 S is special Non-Terminal node.

FA without output

FA with output

Finite Automata

Finite State Machine (Finite Automata)

$\langle S \rangle \Rightarrow \langle \underline{NP1} \rangle \langle VP \rangle$

$\Rightarrow \langle \text{Article} \rangle \langle \text{Noun1} \rangle \langle VP \rangle$

$\Rightarrow \text{The} \langle \text{Noun1} \rangle \langle VP \rangle$

$\Rightarrow \text{The woman} \langle \text{verb} \rangle \langle \text{noun phrase 2} \rangle$

$\Rightarrow \text{The woman ate} \langle \text{article} \rangle \langle \text{Noun2} \rangle$

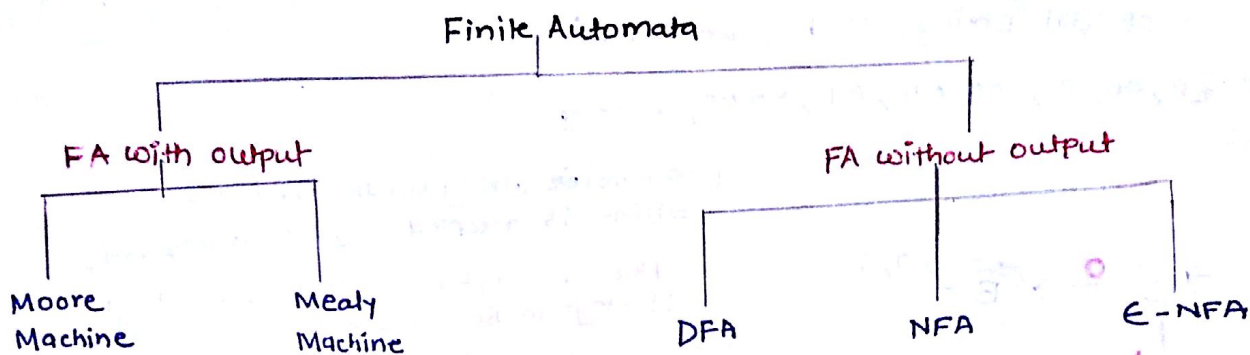
$\Rightarrow \text{The woman ate the fruit.}$

This is called derivation

$\rightarrow$ rewritten as

$\Rightarrow$ directly derives

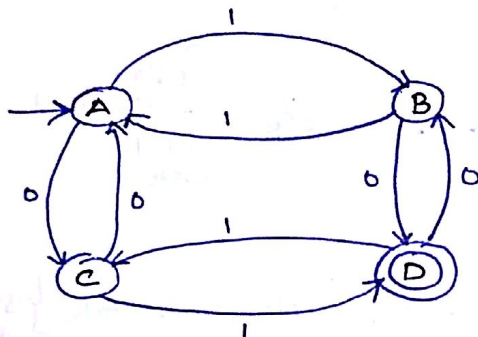
Finite State Machine (Finite Automata)



DFA - Deterministic Finite Automata

- FSM - It is the simplest model of computation
- It has a very limited memory

DFA:-



Circle is known as states $\therefore$ A, B, C, D

edges is transition

labeling of edges is inputs.

A is the initial or starting states of DFA - arrow

D is the final or terminating states of DFA - Double Circle.

Defⁿ:-
 $(Q, \Sigma, q_0, F, \delta)$

Q = set of all states

Σ = inputs

q_0 = start state / initial state

F = set of Final states

δ = transition function from $Q \times \Sigma \rightarrow Q$

$Q = \{A, B, C, D\}$

$\Sigma = \{0, 1\}$

$q_0 = A$

$F = \{D\}$

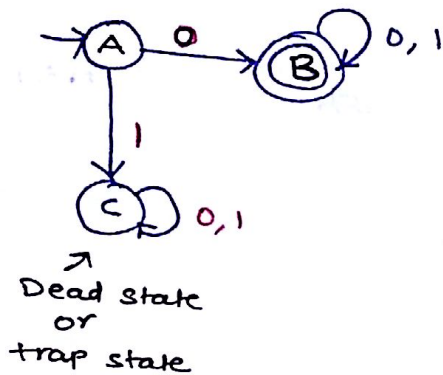
$\delta =$

	0	1
A	C	B
B	D	A
C	A	D
D	B	C

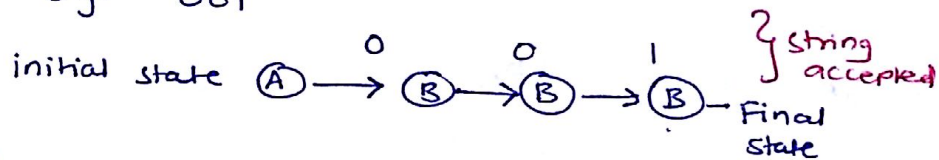
Deterministic Finite Automata (Example-1)

$L_1 =$ set of all strings that start with '0'
 $= \{0, 00, 01, 000, 010, 011, 0000, \dots\}$

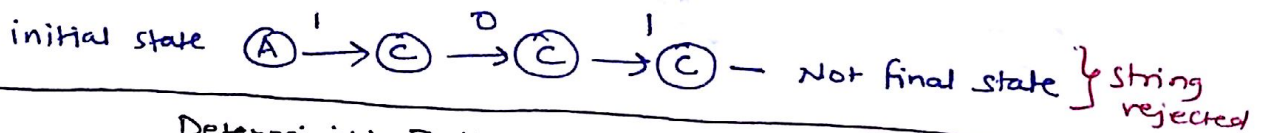
[Whenever string reaches final state string is accepted, else it is rejected]
 (after reaching final state, if we get input it stays in final state only)



E.g. 001 ✓



E.g. 101 ✗

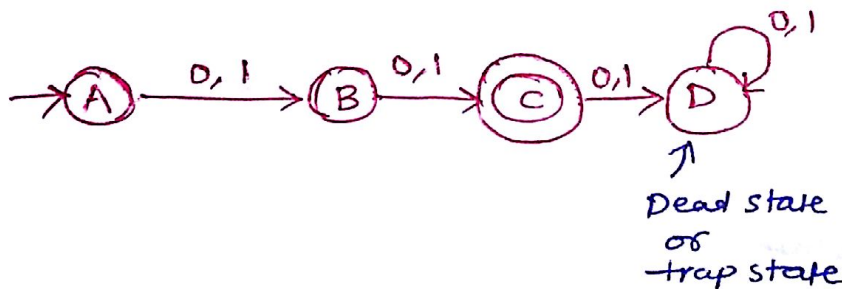


Deterministic Finite Automata (Example-2)

Construct a DFA that accepts sets of all strings over $\{0,1\}$ of length 2

$\Sigma = \{0,1\}$

$L = \{00, 01, 10, 11\}$



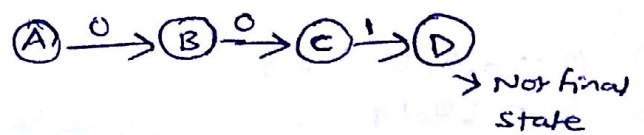
E.g. 00 ✓



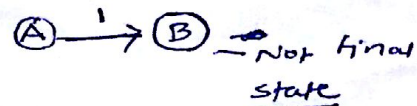
E.g. 10 ✓



E.g. 001 ✗



E.g. 1 ✗



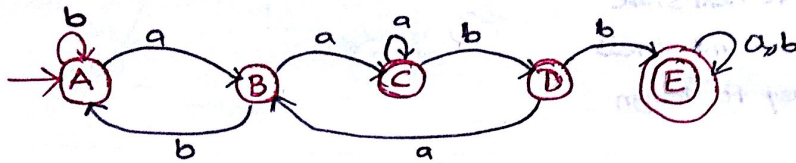
Deterministic Finite Automata (Example-3)

Construct a DFA that accepts any strings over $\{a, b\}$ that does not contain the string aabb in it.

$$\Sigma = \{a, b\}$$

Try to design a simpler problem.

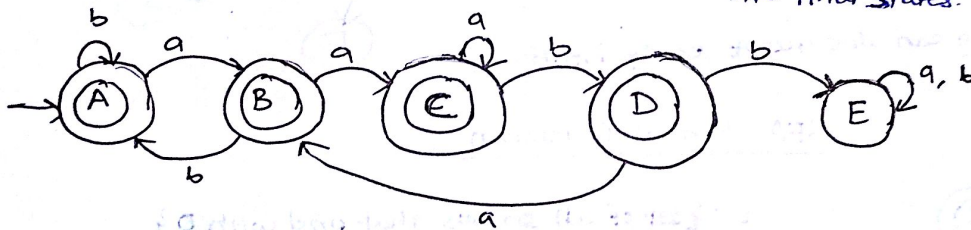
Let us construct a DFA that accepts all strings over $\{a, b\}$ that contains the string aabb in it.



a a a b b

a a b a ...

- Flip the states
- make the final state into non-final state and
- make the non-final states into final states.



Since E is not a final state string is not accepted.

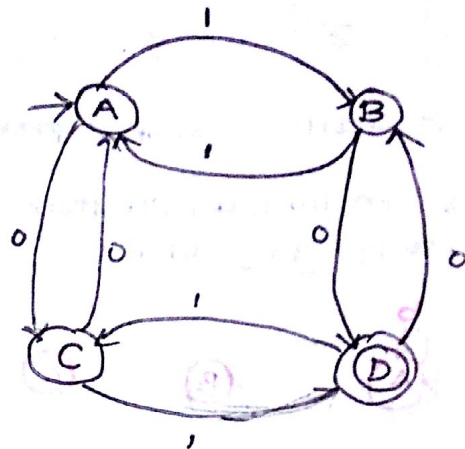
NFA - Non-Deterministic Finite Automata

Deterministic Finite Automata



DETERMINISM

- » In DFA, given the current state we know ~~that~~ what the next state will be.
- » It has only one unique next state
- » It has no choices or randomness
- » It is simple and easy to design

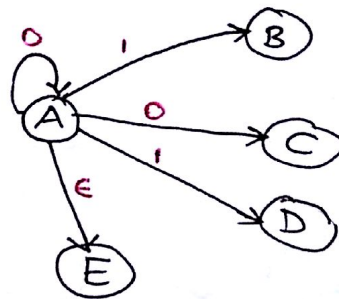


Non-deterministic Finite Automata

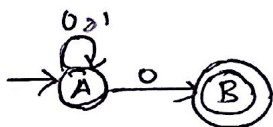


NON-DETERMINISM

- » In NFA, given the current state there could multiple next states.
- » The next state may be chosen at random
- » All the next states may be chosen in parallel.
- ε :- This ^{state} string can also accept empty input



NFA - Formal Definition



$L = \{\text{set of all strings that end with } 0\}$

$(Q, \Sigma, q_0, F, \delta)$

- | | |
|--|--------------|
| $Q =$ set of states | - $\{A, B\}$ |
| $\Sigma =$ inputs | - $\{0, 1\}$ |
| $q_0 =$ start state / initial state | - A |
| $F =$ set of final states | - B |
| $\delta = Q \times \Sigma \rightarrow Q$ | - ? |

$$A \times 0 \rightarrow A$$

$$A \times 1 \rightarrow B$$

$$B \times 0 \rightarrow \phi$$

$$B \times 1 \rightarrow \phi$$

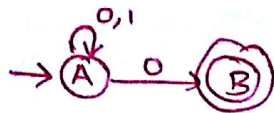
$$A \xrightarrow{1} A, B, AB, \phi \quad 2^{2 \rightarrow 4}$$

3 states - A, B, C

$$A \xrightarrow{1} A, B, C, AB, AC, BC, ABC, \phi$$

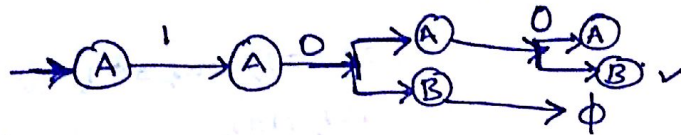
$$2^3 \rightarrow 8$$

NFA - Example-1

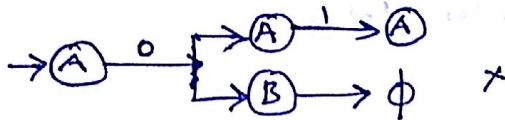


$L = \{ \text{set of all strings that end with 0} \}$

e.g. 100



e.g. 01



✓ [If there is any way to run the machine that ends in any set of states out of which at least one state is a final state, then the NFA accepts.]

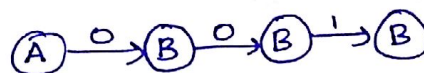
NFA - EXAMPLES - 2

$L = \{ \text{set of all strings that starts with 0} \}$

$= \{ 0, 00, 01, 000, \dots \}$



e.g. 001 ✓



e.g. 101 x



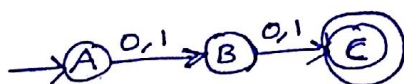
Dead configuration

construct a NFA that accepts sets of all strings over $\{0,1\}$ of length 2

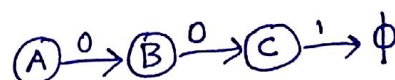
$\Sigma = \{0,1\}$

$L = \{00, 01, 10, 11\}$

e.g. 00 ✓

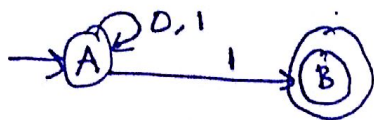


e.g. 001 x



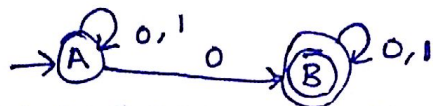
NFA - EXAMPLES 3

EX 1) $L_1 = \{\text{set of all strings that ends with '1'}\}$

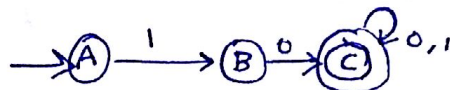


(0, 1, 001, 000, 0*1, 1)
101, 1101,)

EX 2) $L_2 = \{\text{set of all strings that contain '0'}\}$



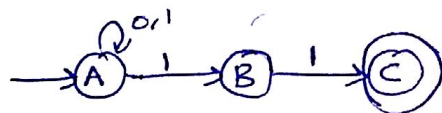
EX 3) $L_3 = \{\text{set of all strings that starts with '10'}\}$



EX 4) $L_4 = \{\text{set of all strings that contain '01'}\}$



EX 5) $L_5 = \{\text{set of all strings that ends with '11'}\}$



Assignment:- If you were to construct the equivalent DFAs for the above NFAs then tell me how many minimum number of states would you use for the construction of each of the DFAs.

- 1) 2
- 2) 2
- 3) 4
- 4) 3
- 5) 3

Conversion of NFA to DFA

Every DFA is an NFA, but not vice versa.

But there is an equivalent DFA for every NFA.

DFA

$$\delta: Q \times \Sigma \rightarrow Q$$

NFA

$$\delta: Q \times \Sigma \rightarrow 2^Q$$

$$NFA \approx DFA$$

$L = \{ \text{Set of all strings over } \{0,1\} \text{ that starts with '0'} \}$

$$\Sigma = \{0,1\}$$

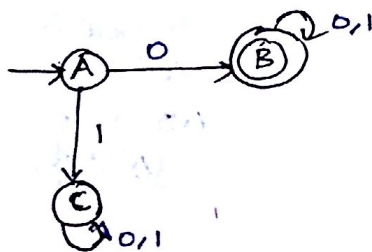
NFA



	0	1
A	B	ϕ
B	B	B

dead configuration

DFA



	0	1
A	B	C
B	B	B
C	C	C

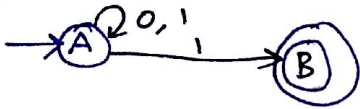
C - dead state / trap state

Conversion of NFA to DFA Example - 2

$L = \{ \text{set of all strings over } (0,1) \text{ that ends with '1'} \}$

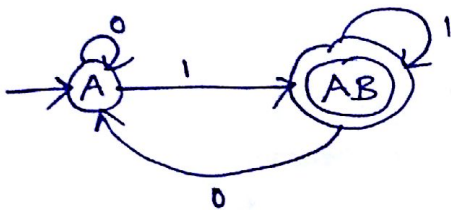
$\Sigma = 0, 1$

NFA



	0	1
A	{A}	{A, B}
B	ϕ	ϕ

DFA

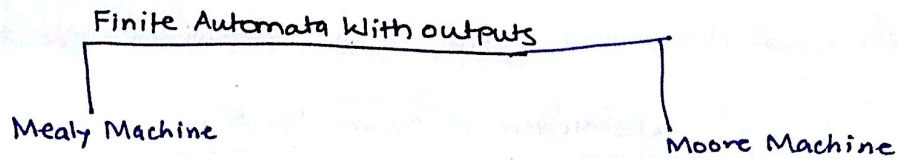


	0	1
A	{A}	{AB}
AB	{A}	{AB}

AB - single state

for calculating
 AB just
 Union set of
 A and B
 $A \cup B$
 AB on 0
 $\{A \cup \phi\} = \{A\}$

✓ (Subset construction Method)



$(Q, \Sigma, \Delta, \delta, \lambda, q_0)$

where

Q = finite set of sets

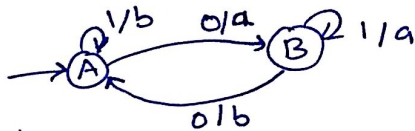
Σ = finite non-empty set of Input Alphabets

Δ = The set of output Alphabets

δ = Transition function $Q \times \Sigma \rightarrow Q$

λ = output function: $\Sigma \times Q \rightarrow \Delta$

q_0 = Initial state / start state



output is dependent on input and state

$(Q, \Sigma, \Delta, \delta, \lambda, q_0)$

where

Q = Finite set of states

Σ = Finite non-empty set of Input alphabets

Δ = The set of output Alphabets

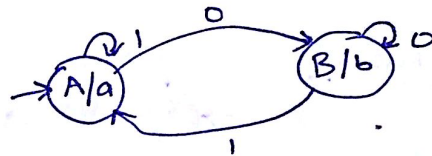
δ = Transition function:

$Q \times \Sigma \rightarrow Q$

λ = output function:

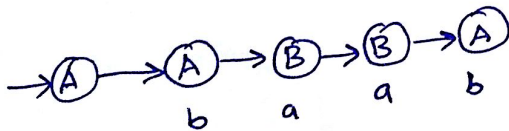
$Q \rightarrow \Delta$

q_0 = Initial state / start state



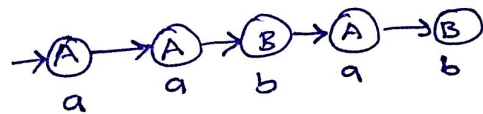
output dependent on State only.

E.g. 1010



length of output string
 $n \rightarrow n$

E.g. 1010



length of output string.
 $n \rightarrow n+1$

Construction of Mealy Machine

Ex-1) Construct a Mealy Machine that produces the 1's complement of any binary input string.

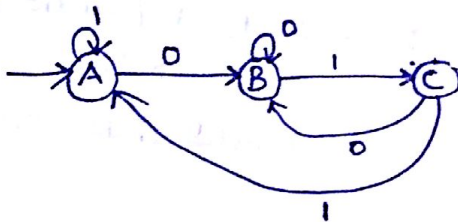


1's complement

1	0	0	0
0	1	0	1

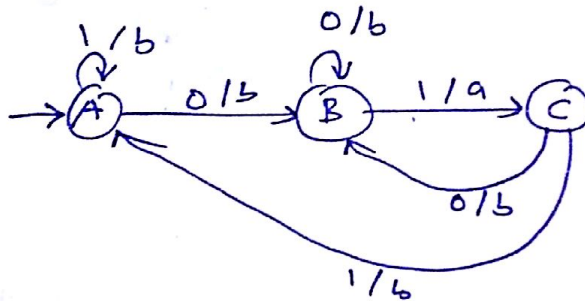
Ex-2) Construct a Mealy Machine that prints 'a' whenever the sequence '01' is encountered in any input binary string

DFA



$\Sigma = \{0, 1\}$ $\Delta = \{a, b\}$

Mealy Machine



e.g.

0	1	1	0
b	a	b	b

✓

1	0	0	0
b	b	b	b

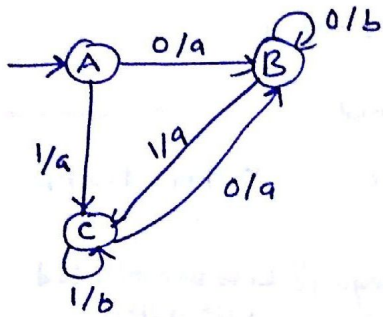
x

Conversion of Mealy Machine to Moore Machine

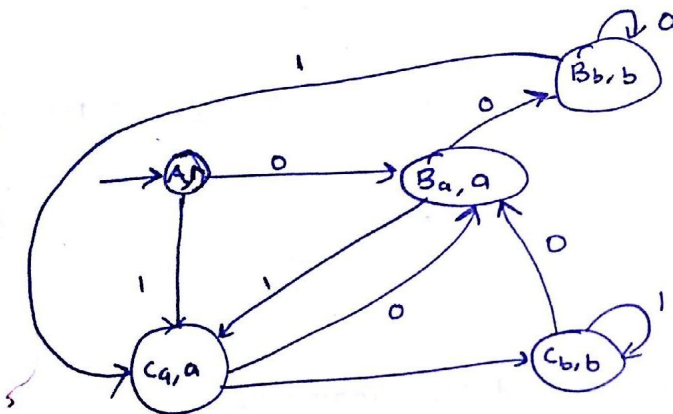
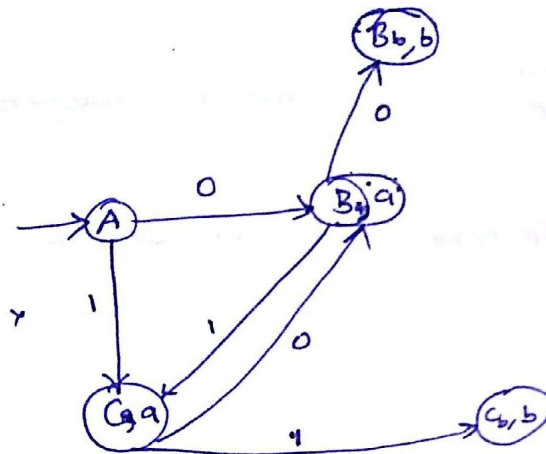
Convert the following Mealy Machine to its equivalent Moore Machine

$$\Sigma = \{0, 1\}$$

$$\Delta = \{a, b\}$$



Moore Machine



Moore $\rightarrow$ Mealy $\Rightarrow$ No of states were same

Mealy $\rightarrow$ Moore $\Rightarrow$ No. of states increased

$\downarrow$
 $\frac{x}{\text{states}}$ and $\frac{y}{\text{outputs}}$ $\Rightarrow (x \times y)$ no. of states at maximum.

Noam Chomsky gave a Mathematical model of Grammar which is effective for Writing Computer languages.

The four types of Grammar according to Noam Chomsky are:

Grammar Type	Grammar Accepted	Language Accepted	Automaton
TYPE-0	Unrestricted Grammar	Recursively Enumerable Language	Turing Machine
TYPE-1	Context Sensitive Grammar	Context Sensitive Language	Linear Bounded Automaton
TYPE-2	Context Free Grammar	Context Free Language	Pushdown Automaton
TYPE-3	Regular Grammar	Regular Language	Finite State Automaton

Grammar:-

A Grammar 'G' can be formally described using 4 tuples as $G = (V, T, S, P)$ where,

V = Set of Variables or Non-Terminal Symbols

T = Set of Terminal Symbols

S = Start Symbol

P = production rules for Terminals and Non-Terminals

A production rule has the form $\alpha \rightarrow \beta$ where α and β are strings on $V \cup T$ and at least one symbol of α belongs to V .

Example:- $G = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$

$V = \{S, A, B\}$

$T = \{a, b\}$

$S = S$

$P = \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\}$

Eg:-

$S \rightarrow AB$

$\rightarrow aB$

$\rightarrow \underline{ab}$

Regular Grammar

Regular Grammar can be divided into two types:

Right Linear Grammar

A grammar is said to be Right Linear if all productions are of the form

$$A \rightarrow xB$$

$$A \rightarrow x$$

where $A, B \in V$ and $x \in T$

Eg:- $S \rightarrow abS \mid b \rightarrow$ Right linear

$S \rightarrow Sbb \mid b \rightarrow$ left linear

Left Linear Grammar

A grammar is said to be Left Linear if all productions are of the form

$$A \rightarrow Bx$$

$$A \rightarrow x$$

where $A, B \in V$ and $x \in T$.

$S \rightarrow Sbb \mid b \rightarrow$ left linear

TYPE 3 : Regular Grammar

Right linear

Left linear

Context free Grammar / Language ()

In formal language theory, a context free Language is a language generated by some Context free Grammar.

The set of all CFL is identical to the set of languages accepted by pushdown Automata.

Context Free Grammar is defined by 4 tuples as $G = \{V, \Sigma, S, P\}$ where

V = Set of Variables or Non-Terminal Symbols

Σ = set of terminal symbols

S = start symbol

P = production rule

Context Free Grammar has production Rule of the form $A \rightarrow \alpha$

where, $\alpha = \{V \cup \Sigma\}^*$ and $A \in V$

Example :- For generating a language that generates equal number of a's and b's in the form $a^n b^n$, the Context Free Grammar will be defined as.

$$G = \{(S, A), (a, b), (S \rightarrow aAb, A \rightarrow aAb | \epsilon)\}$$

Solⁿ :-

$$\begin{aligned} S &\rightarrow aAb \\ &\rightarrow aaAbb \text{ (by } A \rightarrow aAb) \\ &\rightarrow aaaaAbbbb \text{ (by } A \rightarrow aAb) \\ &\rightarrow aaaa\epsilon bbbb \text{ (by } A \rightarrow \epsilon) \\ &\rightarrow aaaa bbbb \\ &\rightarrow a^3 b^3 \Rightarrow a^n b^n \end{aligned}$$

Derivations from a Grammar

The set of all strings that can be derived from a grammar is said to be the LANGUAGE generated from that grammar.

Example 1:- Consider the Grammar $G_1 = (\{S, A\}, \{a, b\}, S, \{S \rightarrow aAb, aA \rightarrow aaAb, A \rightarrow \epsilon\})$

$$\begin{aligned} S &\rightarrow aAb \text{ [by } S \rightarrow aAb] \\ &\rightarrow aaAbb \text{ [by } aA \rightarrow aaAb] \\ &\rightarrow aaaaAbbbb \text{ [by } aA \rightarrow aaAb] \\ &\rightarrow aaaa\epsilon bbbb \text{ [by } A \rightarrow \epsilon] \\ &\rightarrow aaaa bbbb \end{aligned}$$

Example 2:- $G_2 = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow a, B \rightarrow b\})$

$$\begin{aligned} S &\rightarrow AB \\ &\rightarrow ab \text{ [by } A \rightarrow a, B \rightarrow b] \end{aligned}$$

$L(G_2) = \{ab\}$ - The only string generated by Grammar

Example 3:- $G_3 = (\{S, A, B\}, \{a, b\}, S, \{S \rightarrow AB, A \rightarrow aA | a, B \rightarrow bB | b\})$

$\begin{aligned} S &\rightarrow AB \\ &\rightarrow ab \text{ [by } A \rightarrow a, B \rightarrow b] \end{aligned}$	$\begin{aligned} S &\rightarrow \underline{A}B \\ &\rightarrow a\underline{A}bB \\ &\rightarrow qaabb \end{aligned}$	$\begin{aligned} S &\rightarrow AB \\ &\rightarrow aAb \\ &\rightarrow qaab \end{aligned}$
$\begin{aligned} S &\rightarrow AB \\ &\rightarrow abB \\ &\rightarrow abb \end{aligned}$		

$$\begin{aligned} L(G_3) &= \{ab, a^2b^2, a^2b, ab^2, \dots\} \\ &= \{a^m b^n \mid m \geq 0 \text{ and } n \geq 0\} \end{aligned}$$

Regular Expressions are used for representing certain sets of strings in an algebraic fashion

Rules for Regular Expression

- 1) Any terminal symbol i.e. Symbols $\in \Sigma$ including Λ and ϕ are regular expressions. $a, b, c, \dots, \Lambda, \phi$
- 2) The union of two regular expressions is also a regular expression: $R_1, R_2, (R_1 + R_2)$
- 3) The Concatenation of two regular expressions is also a regular expression. $R_1, R_2 \rightarrow (R_1 R_2)$
- 4) The iteration (or closure) of a regular expression is also a regular expression. $R \rightarrow R^*$
- 5) The regular expression over Σ are precisely those obtained recursively by the application of the above rules once or several times. $a^* = \Lambda, a, aa, aaa, \dots$

Regular Expression - Examples

Describe the following sets as Regular Expressions

1) $\{0, 1, 2\}$

0 or 1 or 2 $(+ \rightarrow \text{OR})$

$R = 0 + 1 + 2$

2) $\{\Lambda, ab\}$

$R = \Lambda ab$

3) $\{abb, a, b, bba\}$

abb or a or b or bba

$R = abb + a + b + bba$

4) $\{\Lambda, 0, 00, 000, \dots\}$

closure of 0

$R = 0^*$

5) $\{1, 11, 111, 1111, \dots\}$

closure of 1.

~~$R = 1^+$~~ $R = 1^+$

. it looks like a closure, but Λ is absent.

Identities of Regular Expression

$$1) \phi + R = R$$

$$2) \phi R + R\phi = \phi$$

$$3) \epsilon R = R\epsilon = R$$

$$4) \epsilon^* = \epsilon \text{ and } \phi^* = \epsilon$$

$$5) R + R = R$$

$$6) R^* R^* = R^*$$

$$7) RR^* = R^*R$$

$$8) (R^*)^* = R^*$$

$$9) \epsilon + RR^* = \epsilon + R^*R = R^*$$

$$10) (PQ)^*P = P(QP)^*$$

$$11) (P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$$

$$12) (P + Q)R = PR + QR \text{ and}$$

$$R(P + Q) = RP + RQ$$

Ardens Theorem

IF P and Q are two Regular Expressions over Σ , and if P does not contain ϵ , then the following equation in R given by $R = Q + RP$ has a unique solution i.e. $R = QP^*$

$$R = Q + RP \longrightarrow \textcircled{1}$$

$$R = QP^*$$

$$= Q + QP^*P$$

$$= Q(\epsilon + P^*P)$$

$$[\epsilon + R^*R = R^*]$$

$$= QP^*$$

proved

$$R = Q + RP$$

$$= Q + [Q + RP]P$$

$$= Q + QP + RP^2$$

$$= Q + QP + [Q + RP]P^2$$

$$= Q + QP + QP^2 + RP^3$$

$\vdots$

$$= Q + QP + QP^2 + \dots + QP^n + RP^{n+1}$$

$$[R = QP^*]$$

$$= \emptyset [\epsilon + p + p^2 + \dots + p^n + p^{n+1}]$$

$$\underline{\underline{R = \emptyset p^*}}$$

An Example proof using Identities of Regular Expressions

prove that $(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1)$
is equal to $0^*1(0 + 10^*1)^*$.

$$\text{LHS} = (1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1)$$

$$= (1 + 00^*1) [\epsilon + (0 + 10^*1)^*(0 + 10^*1)]$$

$$= (1 + 00^*1)(0 + 10^*1)^*$$

$$\rightarrow \epsilon + R^*R = R^*$$

$$= (\epsilon + 00^*1)(0 + 10^*1)^*$$

$$\rightarrow \epsilon \cdot R = R$$

$$= (\epsilon + 00^*)1(0 + 10^*1)^*$$

$$= 0^*1(0 + 10^*1)^*$$

$$\rightarrow \epsilon + R^*R = R^*$$

$$= \text{R.H.S.}$$

Designing Regular Expressions --- Examples (Part-1)

Design Regular Expression for the following languages over $\{a, b\}$

- 1) Language accepting strings of length exactly 2.
- 2) Language accepting strings of length atleast 2
- 3) Language accepting strings of length atmost 2

Solⁿ.

$$1) L_1 = \{aa, ab, ba, bb\}$$

$$\begin{aligned} R &= aa + ab + ba + bb \\ &= a(a+b) + b(a+b) \\ &= (a+b)(a+b) \end{aligned}$$

$\therefore$

$$2) L_2 = \{aa, ab, ba, bb, aa, \dots\}$$

$$R = (a+b)(a+b)(a+b)^*$$

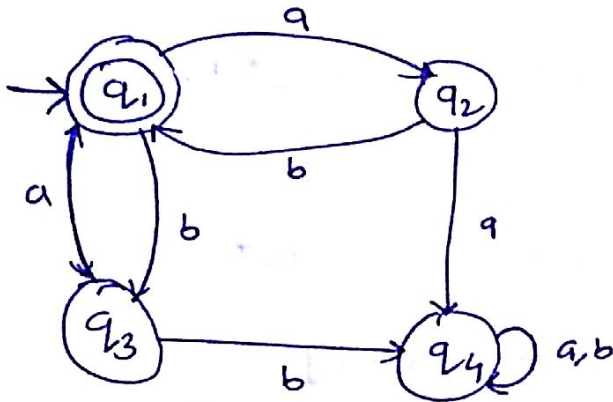
.... two or more than two

$$3) L_3 = \{\epsilon, a, b, aa, ab, ba, bb\}$$

$$\begin{aligned} R &= \epsilon + a + b + aa + ab + ba + bb \\ &= (\epsilon + a + b)(\epsilon + a + b) \end{aligned}$$

Designing Regular Expression - Examples - ②

Find the Regular Expression for the following DFA



$$q_1 = \epsilon + q_2 b + q_3 a \rightarrow \textcircled{1}$$

$$q_2 = q_1 a \rightarrow \textcircled{2}$$

$$q_3 = q_1 b \rightarrow \textcircled{3}$$

$$q_4 = q_2 a + q_3 b + q_4 a + q_4 b \rightarrow \textcircled{4}$$

$$\textcircled{1} \quad q_1 = \epsilon + q_2 b + q_3 a$$

Putting values of q_2 and q_3 from $\textcircled{2}$ and $\textcircled{3}$

$$q_1 = \epsilon + q_1 a b + q_1 b a$$

$$R \rightarrow q_1 = \epsilon + \underbrace{q_1}_{Q} (\underbrace{ab}_{R} + \underbrace{ba}_{P})$$

$$q_1 = \epsilon (ab + ba)^*$$

$$q_1 = (ab + ba)^*$$

$\hookrightarrow$ Regular Expression ..

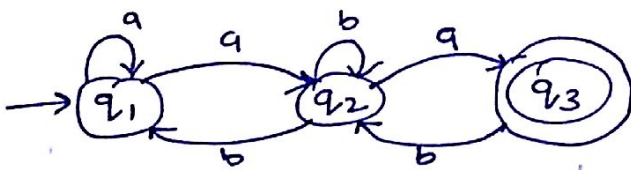
$$\dots R = Q + RP$$

$$\dots R = QP^* \quad \text{Ardens theorem}$$

$$\dots \epsilon R = R$$

Designing Regular Expression - Examples - ②

Find the Regular Expression for the following NFA



$$q_3 = q_2 a \rightarrow \textcircled{1}$$

$$q_2 = q_1 a + q_2 b + q_3 b \rightarrow \textcircled{2}$$

$$q_1 = \epsilon + q_1 a + q_2 b \rightarrow \textcircled{3}$$

$$\textcircled{1} \rightarrow q_3 = q_2 a$$

$$= (q_1 a + q_2 b + q_3 b) a \quad \dots \text{by eqn } \textcircled{2}$$

$$= q_1 a a + q_2 b a + q_3 b a \rightarrow \textcircled{4}$$

$$\textcircled{2} \quad q_2 = q_1 a + q_2 b + q_3 b \quad \text{putting value of } q_3 \text{ from } \textcircled{1}$$

$$= q_1 a + q_2 b + (q_2 a) b$$

$$= q_1 a + q_2 b + q_2 a b$$

$$q_2 = \underbrace{q_1 a}_{Q} + \underbrace{q_2}_{Q} (\underbrace{b}_{R} + \underbrace{ab}_{P})$$

$$\dots R = Q + RP$$

$$\dots R = QP^* \quad \text{Ardens Theorem}$$

$$q_2 = (q_1 a)(b+ab)^* \rightarrow (5)$$

③ →

$$q_1 = \epsilon + q_1 a + q_2 b$$

Putting value of q_2 from (5)

$$q_1 = \epsilon + q_1 a + ((q_1 a)(b+ab)^*)b$$

$$q_1 = \underbrace{\epsilon}_R + \underbrace{q_1}_Q \underbrace{a}_R + \underbrace{((q_1 a)(b+ab)^*)}_P b$$

$$R = Q + P$$

$$R = QR^*$$

$$q_1 = \epsilon ((a + a(b+ab)^*)b)^*$$

$$\epsilon \cdot R = R$$

$$q_1 = (a + a(b+ab)^*)^* b \rightarrow (6)$$

Final state (q_3)

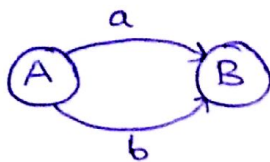
$$q_3 = q_2 a$$

$$q_3 = (q_1 a)(b+ab)^* a \quad \text{Putting value of } q_2 \text{ from (5)}$$

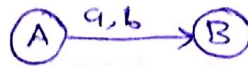
$$q_3 = (a + a(b+ab)^*)^* b a (b+ab)^* a \quad \text{Putting value of } q_1 \text{ from (6)}$$

conversion of Regular Expression to Finite Automata
Rules for designing

(a + b)



or



(a.b)



a^*



Conversion of Regular Expression to Finite Automata - Examples (Part I)

Convert the following Regular Expressions to their equivalent Finite Automata :

1) ba^*b

2) $(a+b)c$

3) $a(bc)^*$

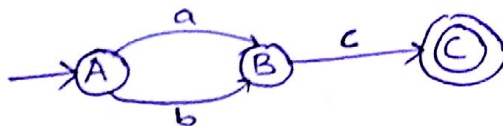
1) ba^*b

$bb, bab, baab, \dots$



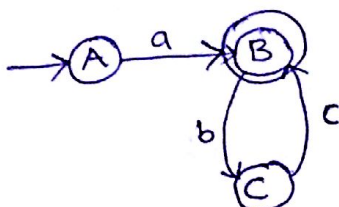
2) $(a+b)c$

$ac, \checkmark$
 $bc, \checkmark$



3) $a(bc)^*$

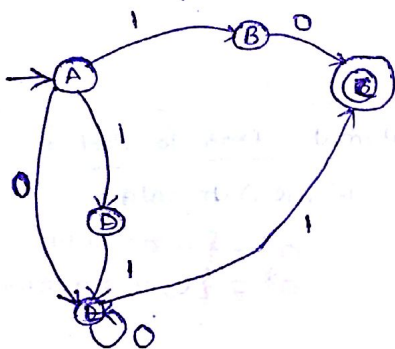
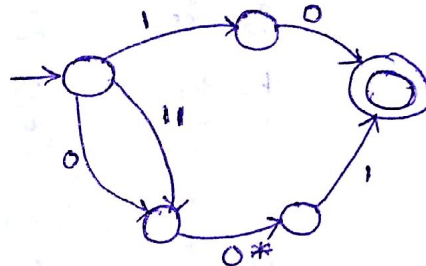
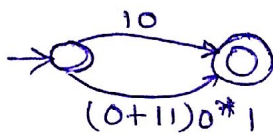
$a, \checkmark$ $abc, \checkmark$ $abcabc, \checkmark$ $abcabcabc, \checkmark$



Conversion of Regular Expression to Finite Automata - Example (Part-3)

Convert the following Regular Expression to its equivalent Finite Automata:

$$10 + (0+11)0^*1$$



Equivalence of two Finite Automata

Steps to identify equivalence

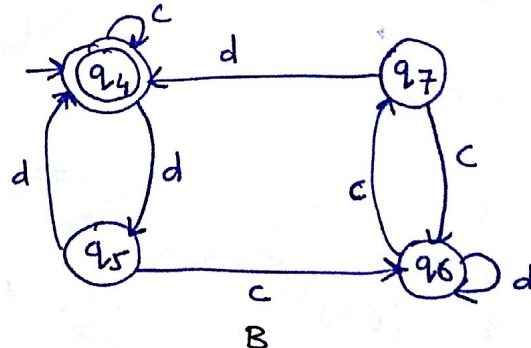
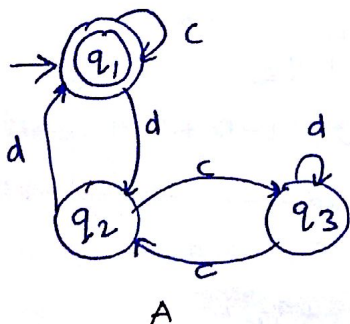
- 1) For any pair of states $\{q_i, q_j\}$ the transition for input $a \in \Sigma$ is defined by $\{q_a, q_b\}$.

where $\delta\{q_i, a\} = q_a$ and $\delta\{q_j, a\} = q_b$

The two automata are not equivalent if for a pair $\{q_a, q_b\}$ one is INTERMEDIATE state and the other is FINAL state.

- 2) If initial state is Final state of one Automaton, then in second automaton also Initial state must be Final state for them to be equivalent.

E.g.



<u>States</u>	<u>c</u>	<u>d</u>
(q_1, q_4)	(q_1, q_4)	(q_2, q_5)
	$\downarrow_{FS} \downarrow_{FS}$	$\downarrow_{IS} \downarrow_{IS}$
(q_2, q_5)	(q_3, q_6)	(q_1, q_4)
	$\downarrow_{IS} \downarrow_{IS}$	$\downarrow_{FS} \downarrow_{FS}$
(q_3, q_6)	(q_2, q_7)	(q_3, q_6)
	$\downarrow_{IS} \downarrow_{IS}$	$\downarrow_{IS} \downarrow_{IS}$
Newly formed $\rightarrow (q_2, q_7)$	(q_3, q_6)	(q_1, q_4)
	$\downarrow_{IS} \downarrow_{IS}$	$\downarrow_{FS} \downarrow_{FS}$

A and B are equivalent

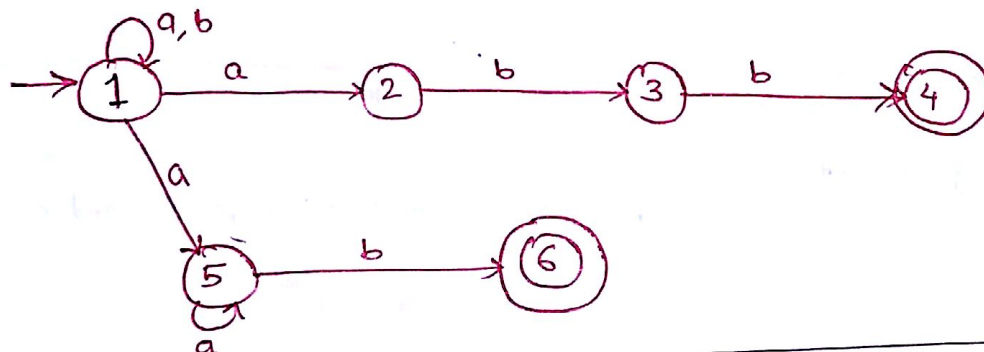
Conversion of Regular Expression to Finite Automata - Examples Part-2)

Convert the following Regular Expression to its equivalent Finite Automata:

$(a|b)^*(abb|a^*b)$

[... (+ & | are same)]

$a^+ = \{a, aa, aaa, \dots\}$
 $a^* = \{\epsilon, a, aa, aaa, \dots\}$



Designing Regular Expression - Examples (Part-4)

(When there are Multiple Final States)



Find the Regular Expression for the following DFA

Final state q_1

① $\rightarrow$

$$\begin{aligned}
 q_1 &= \epsilon + q_1 0 & \text{--- } R &= Q + RP \\
 R &= \epsilon + R 0 & \text{--- } R &= QP^* \\
 q_1 &= (\epsilon 0)^* & \text{--- } \epsilon \cdot R &= R^* \\
 q_1 &= 0^* & \rightarrow & \text{④}
 \end{aligned}$$

$$q_1 = \epsilon + q_1 0 \rightarrow \text{①}$$

$$q_2 = q_1 1 + q_2 1 \rightarrow \text{②}$$

$$q_3 = q_2 0 + q_3 0 + q_3 1 \rightarrow \text{③}$$

Final state q_2

② $\rightarrow$

$$q_2 = q_1 1 + q_2 1$$

$$\begin{aligned}
 q_2 &= 0^* 1 + q_2 1 & \text{--- by putting value of } q_1 \text{ from ④} \\
 R &= 0^* 1 + R 1 & \text{--- } R &= Q + RP \\
 R &= 0^* 1(1)^* & \text{--- } R &= QP^*
 \end{aligned}$$

$$q_2 = 0^* 1(1)^*$$

$R = \text{union of both the final states}$

$$= 0^* + 0^*1(1)^*$$

$$= 0^* + 0^*11^*$$

$$= 0^*(\epsilon + 11^*) \quad \dots \epsilon + RR^* = R^*$$

$$R = 0^*1^*$$

→ Required Expression.

Pumping Lemma (For Regular Languages)

>> Pumping Lemma is used to Prove that a Language is NOT REGULAR.

>> It cannot be used to prove that a Language is Regular.

If A is a Regular Language, then A has a Pumping Length ' P ' such that any string ' S ' where $|S| \geq P$ may be divided into 3 parts

$S = xyz$ such that the following conditions must be true

$$(1) xy^iz \in A \text{ for every } i \geq 0$$

$$(2) |y| > 0$$

$$(3) |xy| \leq P$$

To prove that a language is not Regular Using PUMPING LEMMA, follow below steps;

(We prove using Contradiction)

- Assume that A is Regular
- It has to have a Pumping Length (say P)
- All strings longer than P can be pumped $|S| \geq P$
- Now find a string ' S ' in A such that $|S| \geq P$
- Divide S into xyz
- show that $xy^iz \notin A$ for some i
- Then consider all ways that S can be divided into xyz
- show that none of these can satisfy all the 3 pumping conditions at the same time
- S cannot be pumped == CONTRADICTION.

Pumping Lemma (For Regular Languages) - Example (Part - I)

Using Pumping Lemma Prove that the language $A = \{a^n b^n \mid n \geq 0\}$ is Not Regular

Proof:-

Assume that A is Regular

Pumping length = p

$$S = a^p b^p$$

x y z

$$\Rightarrow S = a a a a a a b b b b b b$$

$$P = 7$$

Case 1: The y part is in the 'a' part

$$\underbrace{a a a a a a a}_x \underbrace{a a a}_y \underbrace{b b b b b b}_z$$

Case 2: The y is in the 'b' part

$$\underbrace{a a a a a a a}_x \underbrace{b b b b b b}_y \underbrace{b}_z$$

Case 3: The y is in the 'a' and 'b' part

$$\underbrace{a a a a a a a}_x \underbrace{a b b b b b}_y \underbrace{b b b b b b}_z$$

For Case 1

$$x y^i z \Rightarrow x y^2 z$$

$$a a a a a a a a a a b b b b b b$$
$$11 \neq 7$$

→ This string does not lie in our language because No. of a's and b's are not equal

For Case 2:-

$$x y^i z \Rightarrow x y^2 z$$

$$a a a a a a a b b b b b b b b b b$$
$$7 \neq 11$$

→ This string does not lie in our language because No. of a's and b's are not equal.

For case 3:-

$$x y^i z \Rightarrow x y^2 z$$

$$a a a a a a a b b b b b b a a b b b b b b$$

→ This string does not lie in our language because No. of a's and b's are not equal $\neq a^n b^n$

$$|xy| \leq P \quad P=7$$

Case 1:-

The language is Not Regular Using Pumping Lemma

Pumping Lemma for Regular Languages (Example Part - II)

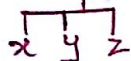
Using Pumping Lemma prove that the language $A = \{yy \mid y \in \{0,1\}^*\}$ is Not Regular

proof:-

0101

Assume that A is Regular

Then it must have a Pumping length = P

$$S = 0^P 1 0^P$$


$$P = 7$$

$$\underbrace{0000000}_x \underbrace{01}_y \underbrace{0000000}_z$$

$$xy^iz \Rightarrow xy^2z$$

$$000000000000100000001$$

$$\notin A$$

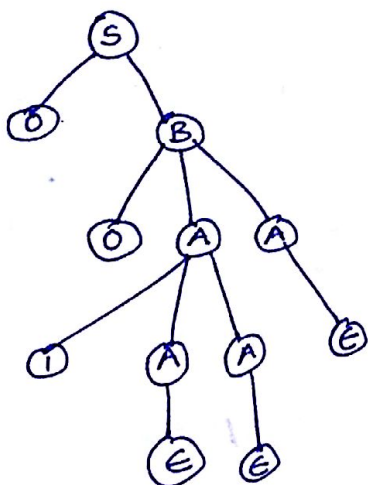
$$|xy| > 0$$

$$|xy| \leq P \quad \text{False} \quad A \text{ is NOT Regular}$$

Derivation Tree

A Derivation Tree or Parse Tree is an ordered rooted tree that graphically represents the semantic information of strings derived from a Context Free Grammar.

Example: For the Grammar $G = \{V, T, P, S\}$ where $S \rightarrow OB$, $A \rightarrow 1AA \mid \epsilon$
 $B \rightarrow OAA$



Root vertex:- Must be labelled by the start symbol
 vertex:- Labelled by Non-Terminal symbols
 Leaves:- Labelled by Terminal symbols or ϵ

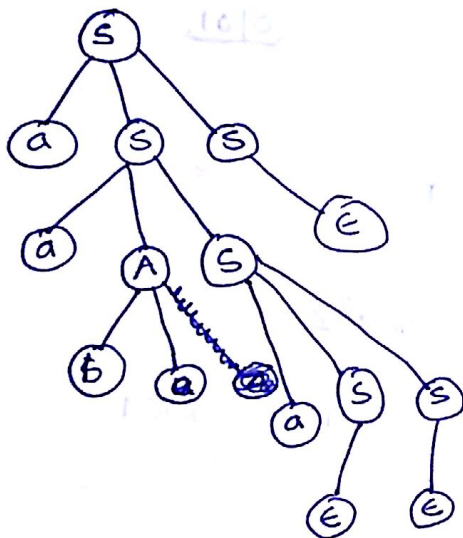
Left Derivation Tree

A Left Derivation Tree is obtained by applying production to the leftmost variable in each step.

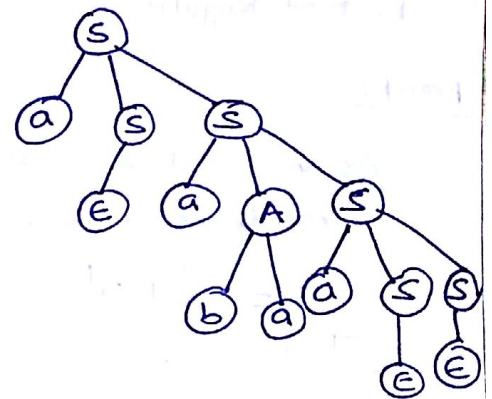
Right Derivation Tree

A Right Derivation Tree is obtained by production to the rightmost variable in each step

E.g. for generating the string aabaa from the Grammar $S \rightarrow aAS \mid aSS \mid \epsilon$, $A \rightarrow sbA \mid ba$



aabaa



aabaa

Ambiguous Grammar

A Grammar is said to be Ambiguous if there exists two or more derivation tree for a string $w \in L$ that means two or more left derivation trees)

Example: $G = (\{S\}, \{a, b, +, *\}, P, S)$ where P consists of

$$S \rightarrow S + S \mid S * S \mid a \mid b$$

The string $a + a * b$ can be generated as:

$$\begin{aligned} S &\rightarrow S + S \\ &\rightarrow a + S \\ &\rightarrow a + S * S \\ &\rightarrow a + a * S \\ &\rightarrow a + a * b \end{aligned}$$

$$\begin{aligned} S &\rightarrow S * S \\ &\rightarrow S + S * S \\ &\rightarrow a + S * S \\ &\rightarrow a + a * S \\ &\rightarrow a + a * b \end{aligned}$$

Thus, this grammar is Ambiguous.

Simplification of Context Free Grammar

In CFG, sometimes all the production rules and symbols are not needed for the derivation of strings. Besides this, there may also be some NULL productions and UNIT productions. Elimination of these productions and symbols is called simplification of CFG.

Simplification consists of the following steps:-

- 1) Reduction of CFG
- 2) Removal of Unit Productions
- 3) Removal of Null Productions.

Reduction of CFG

CFG are reduced in two phases

phase 1:- Derivation of an equivalence grammar ' G_1 ' from the CFG, G , such that each variable ~~derives~~ derives some terminal string

Derivation Procedure:-

Step 1:- Include all symbols w_1 , that derives some terminal and initialize $i = 1$

Step 2:- Include symbols w_{i+1} , that derives w_i

Step 3:- Increment i and repeat step 2, until $w_{i+1} = w_i$

Step 4:- Include all production rules that have w_i in it.

Phase 2:- Derivation of an equivalent grammar G'' , from the GFG, G' , such that each symbol appears in a sentential form.

Derivation Procedures:-

- step 1: Include the start symbol in V_1 and initialize $i=1$
 step 2: Include all symbols V_{i+1} , that can be derived from V_i and include all production rules that have been applied.
 step 3: Increment i and repeat step 2, until $V_{i+1} = V_i$

Example:- Find a reduced grammar equivalent to the Grammar G_1 , having production rules
 $P: S \rightarrow AC \mid B, A \rightarrow a, C \rightarrow c \mid BC, E \rightarrow aA \mid e$

Phase 1:-

$$T = \{a, c, e\}$$

$$W_1 = \{A, C, E\} \text{--- set derive terminals}$$

$$W_2 = \{A, C, E, S\} \text{--- set of element of } W_1$$

$$W_3 = \{A, C, E, S\}$$

$$G' = \{ \{A, C, E, S\}, \{a, c, e\}, P, \{S\} \}$$

$$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c, E \rightarrow aA \mid e$$

phase 2:-

$$V_1 = \{S\}$$

$$V_2 = \{S, A, C\}$$

$$V_3 = \{S, A, C, a, c\}$$

$$V_4 = \{S, A, C, a, c\}$$

$$G'' = \{ \{A, C, S\}, \{a, c\}, P, \{S\} \}$$

$$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c$$

Simplification of Context Free Grammar

Removal of Unit Productions

Any production Rule of the form $A \rightarrow B$ where $A, B \in \text{Non Terminals}$ is called Unit Production.

Procedure for Removal:-

- Step 1:- To remove $A \rightarrow B$, add production $A \rightarrow x$ to the grammar rule whenever $B \rightarrow x$ occurs in the grammar. [$x \in \text{Terminal}$, x can be Null]
- Step 2:- Delete $A \rightarrow B$ from the grammar.
- Step 3:- Repeat from step 1 until all Unit Productions are removed

Example:- Remove Unit Productions from the grammar whose production rule is given by

P: $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow Z|b$, $Z \rightarrow M$, $M \rightarrow N$, $N \rightarrow a$

$Y \rightarrow Z$, $Z \rightarrow M$, $M \rightarrow N$

1) Since $N \rightarrow a$, we add $M \rightarrow a$

P: $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow Z|b$, $Z \rightarrow M$, $M \rightarrow a$, $N \rightarrow a$

2) Since $M \rightarrow a$, we add $Z \rightarrow a$

P: $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow Z|b$, $Z \rightarrow a$, $M \rightarrow a$, $N \rightarrow a$

3) Since $Z \rightarrow a$, we add $Y \rightarrow a$

P: $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow a|b$, $Z \rightarrow a$, $M \rightarrow a$, $N \rightarrow a$

→ From start symbol S we can not reach to Z, M, N .

Remove the unreachable symbols

P: $S \rightarrow XY$, $X \rightarrow a$, $Y \rightarrow a|b$

Simplification of Context Free Grammar

Removal of Null Productions

In a CFG, a Non-Terminal Symbol 'A' is a nullable variable if there is a production $A \rightarrow \epsilon$ or there is a derivation that starts at 'A' and leads to ϵ . (Like $A \rightarrow \dots \rightarrow \epsilon$)

Production for Removal:-

Step 1:- To remove $A \rightarrow \epsilon$, look for all productions whose right side contains A

Step 2:- Replace each occurrence of 'A' in each of these productions with ϵ

Step 3:- Add the resultant productions to the Grammar.

Example:- Remove Null Productions from the following Grammar.

$S \rightarrow ABAC, A \rightarrow aA| \epsilon, B \rightarrow bB| \epsilon, C \rightarrow c$

$A \rightarrow \epsilon, B \rightarrow \epsilon$

1) To eliminate $A \rightarrow \epsilon$

$S \rightarrow ABAC$

$S \rightarrow AB \epsilon C$

$\rightarrow ABC | BAC | BC$

$\rightarrow$

$A \rightarrow aA$

$A \rightarrow a$

New production: $S \rightarrow ABAC | ABC | BAC | BC$

$A \rightarrow aA | a, B \rightarrow bB | \epsilon, C \rightarrow c$

2) To eliminate $B \rightarrow \epsilon$

$S \rightarrow AAC | AC | \epsilon C, B \rightarrow b$

New production: $S \rightarrow ABAC | ABC | BAC | BC | AAC | AC | C$

$A \rightarrow aA | a$

$B \rightarrow bB | b$

$\epsilon \rightarrow c$

Normal Forms

① chomsky Normal form

② Greibach Normal Form

① chomsky Normal form

In chomsky Normal Form (CNF) we have a restriction on length of RHS; which is; elements in RHS should either be two variables or a Terminal.

A CFG is in chomsky Normal Form if the productions are in the following forms:

$$A \rightarrow a$$

$$A \rightarrow BC$$

Where A, B and C are non-terminals and a is a terminal.

Steps to convert a given CFG to Chomsky Normal Form:

Step 1:- If the Start Symbol S occurs on some right side, create a new start Symbol S' and a new production $S' \rightarrow S$.

Step 2:- Remove Null Productions. (Using the Null Production Removal discussed in previous Lecture).

Step 3:- Remove Unit Productions. (———→)

Step 4:- Replace each production $A \rightarrow B_1 \dots B_n$ where $n > 2$, with $A \rightarrow B_1 C$ where $C \rightarrow B_2 \dots B_n$. Repeat this step for all production having two or more symbols on the right side.

Step 5:- If the right side of any production is in the form $A \rightarrow aB$ where 'a' is a terminal and A and B are non-terminals, then the production is replaced by $A \rightarrow XB$ and $X \rightarrow a$.

Repeat this step for every production which is of the form $A \rightarrow aB$.

Conversion of CFG to Chomsky Normal Form

Convert the following CFG to CNF: $P: S \rightarrow ASA | aB, A \rightarrow B | S, B \rightarrow b | \epsilon$

1) Since S appears in RHS, we add new state S' and $S' \rightarrow S$ is added to the production

$$P: S' \rightarrow S, S \rightarrow ASA | aB, A \rightarrow B | S, B \rightarrow b | \epsilon$$

2) Remove the Null Productions: $B \rightarrow \epsilon$ and $A \rightarrow \epsilon$:

After removing $B \rightarrow \epsilon$: $P: S' \rightarrow S, S \rightarrow ASA | aB | a, A \rightarrow B | S | \epsilon, B \rightarrow b$

After removing $A \rightarrow \epsilon$: $P: S' \rightarrow S, S \rightarrow ASA | aB | a | AS | SA | S,$
 $A \rightarrow B | S, B \rightarrow b$

3) Remove the Unit Productions: $S \rightarrow S, S' \rightarrow S, A \rightarrow B, A \rightarrow S$:

After removing $S \rightarrow S$: $P: S' \rightarrow S, S \rightarrow ASA | aB | a | AS | SA, A \rightarrow B | S, B \rightarrow b$

After removing $S' \rightarrow S$: $P: S' \rightarrow ASA | aB | a | AS | SA,$
 $S \rightarrow ASA | aB | a | AS | SA,$
 $A \rightarrow B | S, B \rightarrow b$

After removing $A \rightarrow B$: $P: S' \rightarrow ASA | aB | a | AS | SA,$
 $S \rightarrow ASA | aB | a | AS | SA,$
 $A \rightarrow b | S, B \rightarrow b$

After removing $A \rightarrow S$: $S' \rightarrow ASA | aB | a | AS | SA,$
 $S \rightarrow ASA | aB | a | AS | SA,$
 $A \rightarrow b | \cancel{S} | ASA | aB | a | AS | SA,$
 $B \rightarrow b$

4) Now find out the productions that has more than Two variables in RHS
 $S' \rightarrow ASA, S \rightarrow ASA$ and $A \rightarrow ASA$

After removing these, we get: $P: S' \rightarrow AX | aB | a | AS | SA,$
 $S \rightarrow AX | aB | a | AS | SA,$
 $A \rightarrow b | AX | aB | a | AS | SA,$
 $B \rightarrow b,$
 $X \rightarrow SA$

5) Now change the productions $S' \rightarrow aB$, $S \rightarrow aB$ and $A \rightarrow aB$

Finally we get :

$$P: S' \rightarrow AX|YB|a|AS|SA, (\dots Y \rightarrow a)$$

$$S \rightarrow AX|YB|a|AS|SA,$$

$$A \rightarrow b|AX|YB|a|AS|SA,$$

$$B \rightarrow b,$$

$$* X \rightarrow SA,$$

$$Y \rightarrow a$$

Which is the required chomsky Normal Form for the given CFG.

[Redacted text]

Theory

of

Computation

[Redacted text]

Chay

Greibach Normal Form

A CFG is in Greibach Normal form if the productions are in the following forms:

$$A \rightarrow b$$

$$A \rightarrow bC_1C_2 \dots C_n$$

Where $A, C_1, \dots, C_n$ are Non-Terminals and b is a Terminal.

Steps to convert a given CFG to GNF::

Step 1: check if the given CFG has any Unit Productions or Null Productions and Remove if there are any (using the Unit & Null Productions removal techniques discussed in the previous lecture).

Step 2: Check whether the CFG is already in Chomsky Normal Form (CNF) and convert it to CNF if it is not. (using the CFG to CNF conversion techniques discussed in the previous lecture)

Step 3: change the Names of the Non-Terminal Symbols into some A_i in ascending order of i

Example :-

$$S \rightarrow CA \mid BB$$

$$B \rightarrow b \mid SB$$

$$C \rightarrow b$$

$$A \rightarrow a$$

Replace S with A_1

C with A_2

A with A_3

B with A_4

We get:

$$A_1 \rightarrow A_2 A_3 \mid A_4 A_4$$

$$A_4 \rightarrow b \mid A_1 A_4$$

$$A_2 \rightarrow b$$

$$A_3 \rightarrow a$$

Step 4: Alter the rules so that the Non-Terminals are in ascending order, such that If the Production is of the form $A_i \rightarrow A_j X$, then, $i < j$ and should never be $i \geq j$

$$A_4 \rightarrow b \mid \underline{A_1} A_4$$

$$A_4 \rightarrow b \mid \underline{A_2} A_3 A_4 \mid A_4 A_4 A_4$$

$$A_4 \rightarrow b \mid b A_3 A_4 \mid A_4 A_4 A_4$$

↓
Left Recursion.

Step 5: Remove Left Recursion

(Greibach Normal Form)

(Conversion of CFG to CNF - Removal of Left Recursion)

$$A_1 \rightarrow A_2 A_3 \mid A_4 A_4$$

$$A_4 \rightarrow b \mid A_1 A_4 \rightarrow A_4 \rightarrow b \mid b A_3 A_4 \mid A_4 A_4 A_4$$

Left Recursion

$$A_2 \rightarrow b$$

$$A_3 \rightarrow a$$

Step 5:- Remove Left Recursion

Introduce a New Variable to remove the Left Recursion

$$A_4 \rightarrow b \mid b A_3 A_4 \mid A_4 A_4 A_4$$

$$A_4 \rightarrow A_4 A_4 Z \mid A_4 A_4 \dots (\text{new production one with } Z \text{ and one without } Z)$$

$$A_4 \rightarrow b \mid b A_3 A_4 \mid b Z \mid b A_3 A_4 Z \dots (\text{it is in GNF})$$

Now the grammar is:

$$A_1 \rightarrow A_2 A_3 \mid A_4 A_4$$

$$A_4 \rightarrow b \mid b A_3 A_4 \mid b Z \mid b A_3 A_4 Z$$

$$Z \rightarrow A_4 A_4 \mid A_4 A_4 Z$$

$$A_2 \rightarrow b$$

$$A_3 \rightarrow a$$

~~$$A_1 \rightarrow b A_3 \mid b A_4 \mid b A_3 A_4 \mid b Z \mid b A_3 A_4 Z$$~~

$$A_1 \rightarrow b A_3 \mid b A_4 \mid b A_3 A_4 A_4 \mid b Z A_4 \mid b A_3 A_4 Z A_4$$

$$A_4 \rightarrow b \mid b A_3 A_4 \mid b Z \mid b A_3 A_4 Z$$

--- (Replacing $A_2 \rightarrow b$
 $A_4 \rightarrow b$)

$$Z \rightarrow b A_4 \mid b A_3 A_4 A_4 \mid b Z A_4 \mid b A_3 A_4 Z A_4 \mid$$

$$b A_4 \mid b A_3 A_4 A_4 Z \mid b Z A_4 Z \mid b A_3 A_4 Z A_4 Z$$

--- (Replacing $A_4 \rightarrow b$)

$$A_2 \rightarrow b$$

$$A_3 \rightarrow a$$

Pumping Lemma (for Context Free Languages) -

Pumping Lemma (for CFL) is used to prove that a language is NOT Context Free.

Context Free Language

In formal language theory, a context free language is a language generated by some context free grammar.

The set of all CFL is identical to the set of languages accepted by pushdown Automata.

Context Free Grammar is identified by 4 tuples as $G = \{V, \Sigma, S, P\}$ where

V = set of Variables or Non-Terminal Symbols

Σ = set of Terminal Symbols.

S = Start Symbol

P = production Rule.

Context Free Grammar has Production Rule of the form

$$A \rightarrow a$$

where, $a \in \{V \cup \Sigma\}^*$ and $A \in V$

If A is a Context Free Language, then, A has a Pumping Length ' P ' such that any string ' s ', where $|s| \geq P$ may be divided into 5 pieces $S = uvxyz$ such that the following conditions must be true:

- 1) $u v^i x y^i z$ is in A for every $i \geq 0$
- 2) $|vy| > 0$
- 3) $|vxy| \leq P$

To Prove that a Language is Not context Free using Pumping Lemma (for CFL)
Follow the steps below: (We prove using CONTRADICTION)

- Assume that A is context Free
- It has to have a Pumping Length (say P)
- All strings longer than P can be pumped $|s| \geq P$
- Now find a string ' s ' in A such that $|s| \geq P$
- Divide S into $uvxyz$
- Show that $u v^i x y^i z \notin A$ for some i
- Then consider the ways that S can be divided into $uvxyz$
- show that none of these can satisfy all the 3 pumping conditions at the same time.
- S cannot be pumped == CONTRADICTION

Pushdown Automata (Formal Definition)

A pushdown Automata is formally defined by 7 Tuples as show below:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Where,

Q = A finite set of states

Σ = A finite set of Input Symbols

Γ = A finite Stack Alphabet

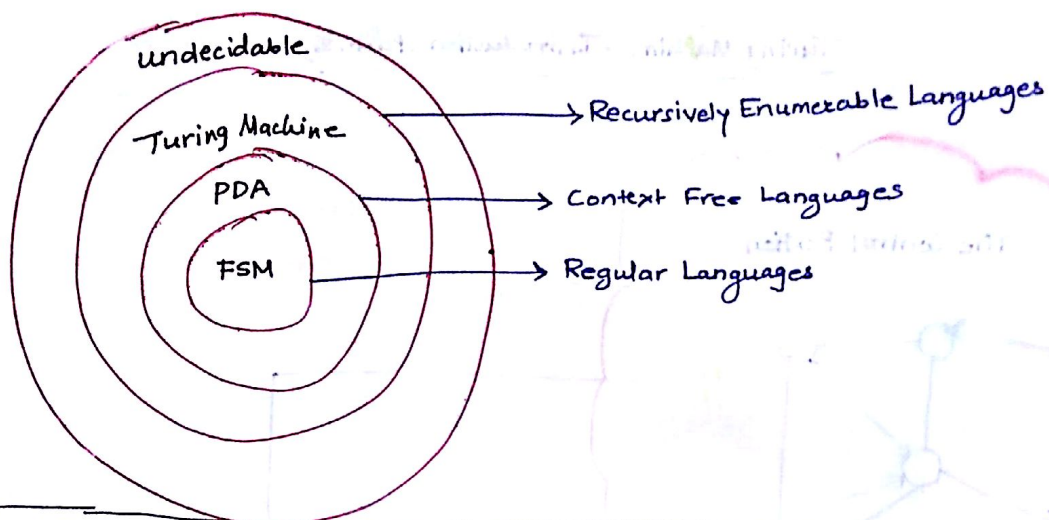
δ = The Transition Function

q_0 = The Start State

Z_0 = The Start Stack Symbol

F = The set of Final / Accepting States

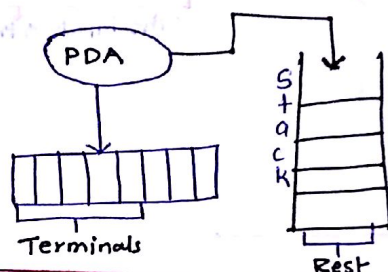
Turing Machine - Introduction (Part-1)



FSM : The Input String

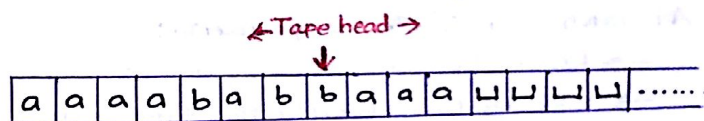
a	a	a	a	b	a	b	b
---	---	---	---	---	---	---	---

PDA : → The Input String
→ A Stack



TURING MACHINE:

→ A Tape

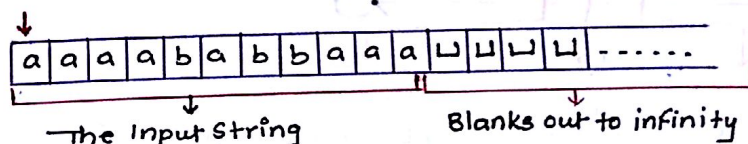


Tape Alphabets: $\Sigma = \{0, 1, a, b, x, Z_0\}$

The Blank U is a Special Symbol $U \notin \Sigma$

The blank is a special symbol used to fill the infinite tape.

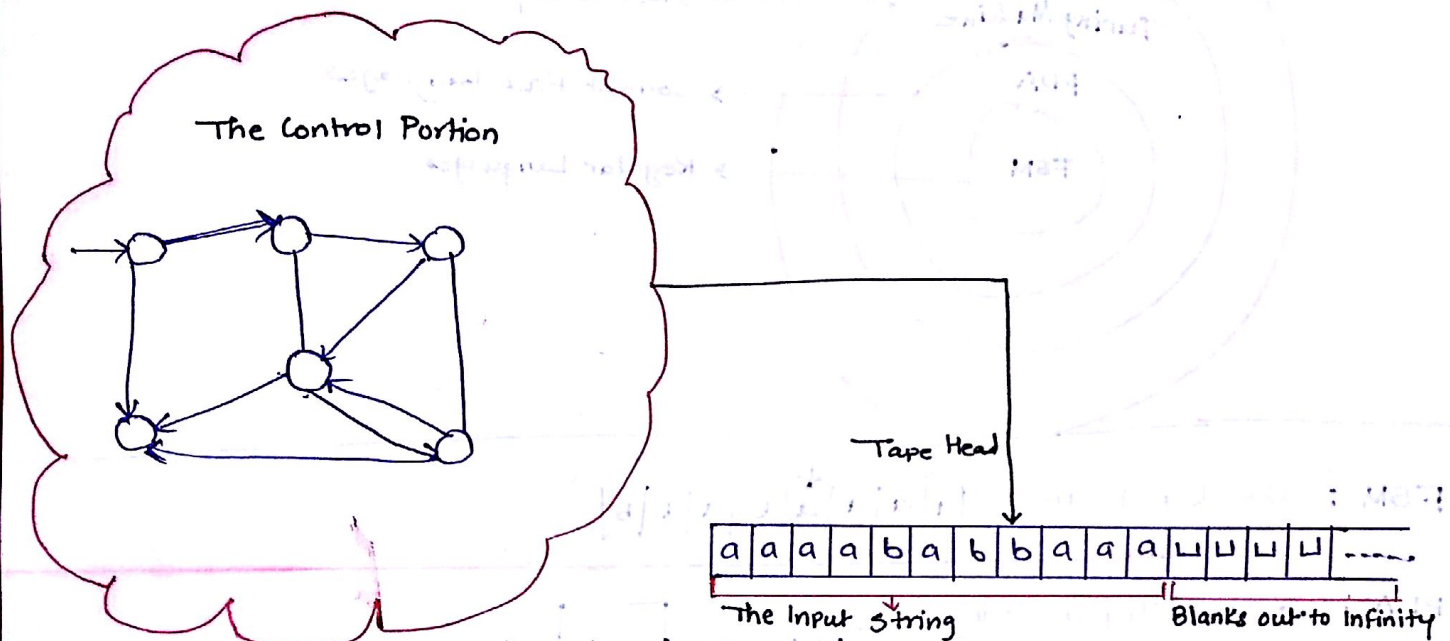
Initial configuration:



Operations on the Tape:

- Read / Scan symbol below the Tape Head.
- Update / Write a symbol below the Tape Head.
- Move the Tape Head one step LEFT
- Move the Tape Head one step RIGHT.

Turing Machine - Introduction (Part-2)



The Control Portion similar to FSM or PDA

The PROGRAM

It is deterministic

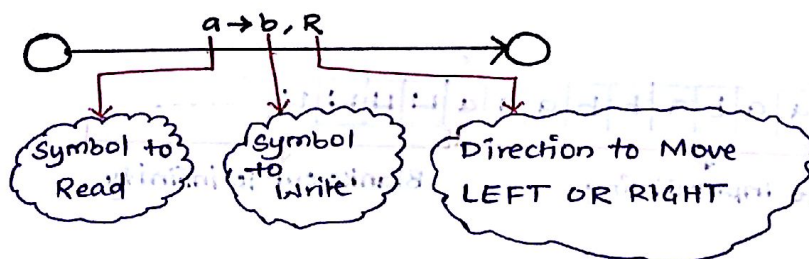
Rules of operation - 1

At each step of the computation:

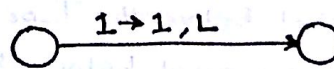
- Read the current symbol
- Update (i.e. write) the same cell
- Move exactly one cell either LEFT OR RIGHT.

If we are at the left end of the tape, and trying to move LEFT, then do not move.

Stay at the left end.



If you don't want to update the cell,
JUST WRITE THE SAME SYMBOL



Rules of operation - 2

- Control is with a sort of FSM
- Initial state
- Final states : (there are two final states)
 - 1) The ACCEPT STATE
 - 2) The REJECT STATE
- Computation can either
 - 1) HALT and ACCEPT
 - 2) HALT and REJECT
 - 3) LOOP (the machine fails to HALT)

Turing Machine (Formal Definition)

A Turing Machine can be defined as a set of 7 tuples
 $(Q, \Sigma, \Gamma, \delta, q_0, b, F)$

$Q \rightarrow$ Non empty set of states

$\Sigma \rightarrow$ Non empty set of symbols

$\Gamma \rightarrow$ Non empty set of Tape symbols ---- (Tau)

$\delta \rightarrow$ Transition function defined as

$$Q \times \Sigma \rightarrow \Gamma \times (R/L) \times Q$$

$q_0 \rightarrow$ Initial state

$b \rightarrow$ Blank symbol

$F \rightarrow$ set of Final states (Accept state & Reject state)

Thus, the production rule of Turing Machine will be written as

$$\delta(q_0, a) \rightarrow (q_1, \gamma, R)$$

Turing's Thesis :

Turing's Thesis states that any computation that can be carried out by Mechanical means can be performed by some Turing Machine

Few arguments for accepting this thesis are:

- i. Anything that can be done on existing digital Computer can also be done by Turing Machine
- ii) No one has yet been able to suggest a problem solvable by what we consider an algorithm, for which a Turing Machine Program cannot be written.

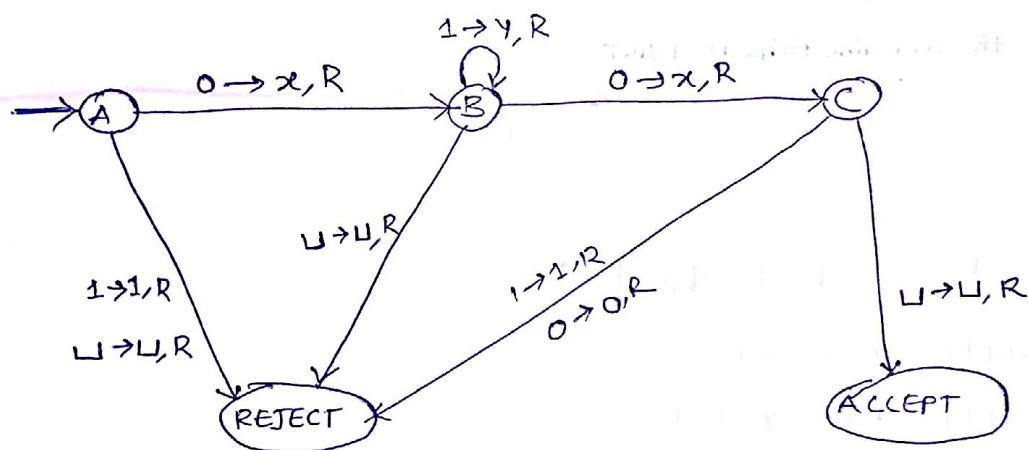
Recursively Enumerable Language:

A Language L and Σ is said to be Recursively Enumerable if there exists a Turing Machine that accepts it.

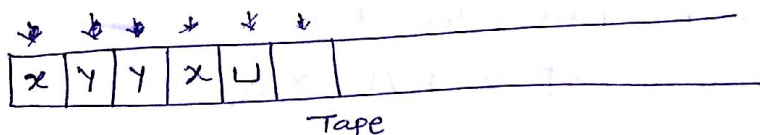
Turing Machine - Example (Part - I)

Design a Turing Machine Which recognizes the language

$$L = 01^*0$$



eg:- 0110 ✓



$$\Sigma = \{0, 1\}$$

$$b = \text{␣}$$

Turing Machine - Example (Part - 2)

Design a Turing Machine which recognizes the language $L = 0^N 1^N$

Algorithm:

- change "0" to "x"
- Move RIGHT to First "1"
- IF None : REJECT
- change "1" to "y"
- Move LEFT to Leftmost "0"
- Repeat the above steps until no more "0"s
- Make sure no more "1"s remain